Manual de referência do NexusDB

Administração desenvolvimento e operações

NexusDB 0.1.1 | Revisão técnica de 11 de novembro de 2026

Este manual reúne os recursos presentes no código local do NexusDB e os procedimentos para usá-los. Destina-se a administradores, desenvolvedores e pesquisadores que precisam instalar o servidor, criar bancos, consultar grafos, controlar acesso e integrar ferramentas. Cada família apresenta pré-requisitos, interfaces, exemplos, efeitos e limites. As referências de implementação permitem conferir os contratos no repositório.

O projeto é experimental. A configuração CPU permite executar os exemplos sem uma GPU. Recursos opcionais dependem das features compiladas e da ativação exigida pelo banco. A organização em conceitos, administração e referência permite consultar uma operação sem seguir todo o manual em sequência.

Organização e sumário

Os capítulos 1 a 4 apresentam conceitos, instalação e acesso. Os capítulos 5 a 8 cobrem consultas e análise. Os capítulos 9 a 12 descrevem conhecimento, funções e ferramentas. Os capítulos 13 a 17 tratam automação, operação e laboratórios.

1 Conceitos arquitetura e contratos....3

2 Instalação configuração e execução....4

3 Conexão HTTP autenticação e respostas....6

4 Bancos usuários e segurança....8

5 Dados consultas e índices....11

6 Algoritmos planejamento e computação....13

7 Histórico projeções e armazenamento em disco....15

8 Vetores GraphRAG e dados espaciais....18

9 Conhecimento memórias e descobertas....20

10 Phronesis e Datalog....23

11 Rotinas NPL....24

12 Agent MCP e grafos executáveis....25

13 Automações e auditoria....28

14 Persistência backup e alta disponibilidade....29

15 Gateway e observabilidade....30

16 Laboratórios e integração de dados....31

17 Diagnóstico configuração e manutenção....33

Os exemplos alteram apenas bancos de demonstração indicados. Os pré-requisitos e limites de cada família devem ser lidos antes da execução.

1 Conceitos arquitetura e contratos

1 1 Nós relacionamentos e propriedades

Um nó representa uma entidade, como Cliente, Produto ou Amostra. Cada nó possui um ID interno, um rótulo e propriedades JSON. Um relacionamento representa uma ligação dirigida, possui tipo e propriedades e referencia os IDs dos dois nós. O motor preserva relações paralelas. A interpretação da direção depende da consulta ou do algoritmo solicitado.

O exemplo comercial segue Cliente → REALIZOU → Pedido → CONTEM → Produto. A data do pedido pode ser uma propriedade ou uma ligação NO_DIA para um nó Dia. A aplicação escolhe o modelo, as unidades monetárias e as regras de identificação de negócio. Um código de produto não substitui automaticamente o ID interno.

Domínio	Nós de exemplo	Relações de exemplo
Comercial	Cliente, Pedido, Produto, Dia	REALIZOU, CONTEM, NO_DIA
Social	Pessoa, Grupo	SEGUE, PARTICIPA
Científico	Amostra, Medicao, Pesquisador	MEDIDA_EM, PRODUZIU, SUPORTA

1 2 Fronteiras do sistema

As interfaces principais são HTTP nativo, procedimentos CALL, API Agent/MCP e API Rust. O servidor autentica a requisição, seleciona o banco e executa o recurso nativo. O REST Gateway é um processo separado que publica operações de leitura previamente configuradas.

O motor padrão mantém dados e índices em memória e persiste por banco em SQLite e arquivos auxiliares. O diário de recuperação e o histórico temporal adicionam registros transacionais. Snapshots/CSR e índices vetoriais são estruturas derivadas versionadas. Um cache é identificado pelo banco e pela revisão ou projeção correspondente.

Há um backend de banco em disco para importação offline, projeções particionadas e BFS. Esse modo possui um conjunto próprio de operações. Uma consulta que exige o motor em memória pode retornar DISK_ONLY_UNSUPPORTED.

A aplicação pode integrar um LLM para gerar uma síntese do contexto recuperado. A API de GraphRAG e o context pack devolvem evidências e texto para consumo externo. A origem das evidências, a execução de análise e a geração de linguagem têm responsabilidades distintas.

1 3 Invariantes globais

Contrato	Consequência prática
Autenticação e banco selecionado	Uma credencial válida em outra instância não autentica aqui
Grants e política Knowledge	Uma chave de integração não amplia as permissões nativas
IDs internos	Use os IDs efetivamente retornados antes de criar relações

Contrato	Consequência prática
Cache versionado	Mudanças no grafo invalidam derivados conforme o caminho de execução
Limites de recursos	Workers HTTP não eliminam quotas de compute e memória
Execução por operação	Não deduza uma transação coletiva a partir de vários statements

NexusDB implementa um subconjunto de consultas Cypher e extensões próprias. Não há declaração de conformidade completa com openCypher 9. O modo HA é experimental e não representa uma implementação completa de consenso Raft. As restrições específicas de cada família são apresentadas nos respectivos capítulos.

1 4 Interfaces e efeitos

Área	Interface de implementação	Efeito
Entrada HTTP	src/data.rs; src/server_runtime.rs	Autentica, autoriza, admite e despacha
Motor	src/model/db.rs	Coordena nós, relações, índices e persistência
Persistência	src/storage.rs; src/recovery_journal.rs	Grava SQLite e diário quando ativado
Análise	src/graph; src/compute	Captura snapshots e calcula resultados com orçamento
Knowledge	src/knowledge	Aplica política, escopos, revisão e evidências
Publicação	crates/nexusdb-rest-gateway	Controla endpoints e chaves de leitura

Referências — Entrada HTTP, motor, persistência e API Rust.

2 Instalação configuração e execução

O NexusDB executa como servidor Rust e admite recursos opcionais de compilação. O primeiro passo operacional é identificar o executável e as features presentes; uma compilação sem flags usa `default = []` e não corresponde necessariamente ao pacote do instalador.

```
nexusdb.exe --build-features
nexusdb.exe run --config C:\NexusDB\nexusdb.conf
```

O comando --build-features anuncia knowledge e phronesis quando compilados. A relação completa de features está em Cargo.toml.

A primeira inicialização precisa de uma senha administrativa fornecida por `NEXUSDB_ADMIN_PASSWORD` ou pelo cofre de bootstrap configurado. Ela deve cumprir a política de senhas. Essa variável inicializa uma instalação nova; não redefine a senha de um administrador existente. Reserve um diretório de dados próprio para cada instância e uma porta livre.

2 1 Configuração

O arquivo básico usa variáveis `CHAVE=valor`. Variáveis já presentes no ambiente prevalecem sobre as carregadas pelo arquivo. O parser também reconhece seções específicas `[compute]`, `[compute.cuda]`, `[agent]`, `[resources]` e

subclasses dos recursos. Não se trata de suporte genérico a qualquer INI: somente as chaves explicitamente reconhecidas são traduzidas para opções nativas.

```
NEXUSDB_HTTP_ADDR=127.0.0.1:7474
NEXUSDB_MODE=SINGLE
NEXUSDB_DATA_ROOT=C:\NexusDB\data
NEXUSDB_RECOVERY_JOURNAL=1

[compute]
backend = "AUTO"
threads = 8
memory_mb = 512
max_jobs = 2
timeout_ms = 300000
```

Os valores de compute acima são escolhas ilustrativas, não uma indicação de que todas as instalações possuem oito threads ou os mesmos limites padrão.

Variável	Padrão confirmado	Uso
NEXUSDB_HTTP_ADDR	127.0.0.1:7474	Endereço do listener nativo
NEXUSDB_HTTP_WORKERS	32	Workers HTTP; intervalo 1–256
NEXUSDB_HTTP_QUEUE	256	Fila limitada de conexões; intervalo 1–4096
NEXUSDB_DATABASE_MAX_JOBS	2	Admissão simultânea de requests identificados por banco
NEXUSDB_DATABASE_MEMORY_MB	256	Reservas gerenciadas agregadas por banco
NEXUSDB_PASSWORD_MIN_LENGTH	15	Mínimo de caracteres para novas senhas

O servidor não termina TLS. Um bind remoto é recusado, salvo opção explícita `NEXUSDB_ALLOW_INSECURE_REMOTE=true`; para publicação remota, configure um proxy TLS e mantenha a porta nativa sob controle. Aumentar workers HTTP não amplia automaticamente os recursos disponíveis para algoritmos. Uma fila cheia pode recusar trabalho; isso deve ser tratado como backpressure, com repetição limitada e atenção especial a gravações cuja confirmação se perdeu.

2 2 Instalação Windows

Para produzir o instalador Windows, são necessários Rust, Visual Studio C++ Build Tools e Inno Setup 6 ou 7. O pacote CUDA também exige Toolkit com `nvcc` na máquina de compilação.

```
build_installer_windows.bat cpu
build_installer_windows.bat cuda --no-pause
```

O instalador solicita knowledge, phronesis, datalog, embedded e spatial-geoparquet. Antes de empacotar, verifica knowledge e phronesis no executável release. O padrão do empacotador é CUDA. O modo CPU permite empacotar sem Toolkit ou DLL CUDA. O script gera o pacote, não o instala nem reinicia o serviço. O instalador configura o serviço sob `LocalService`, binários em `C:\Program Files\NexusDB\bin` e conteúdo mutável em `C:\ProgramData\NexusDB`. Preserva configuração e dados em atualização e desinstalação. O gateway não inicia automaticamente. Python 3.10+ é necessário para a ferramenta de backup distribuída, não para o servidor Rust.

Fontes: features de compilação, entrada e argumentos, configuração e execução, instalador.

2 3 Serviço e encerramento

Os comandos service install, start, stop, restart, status e uninstall administram o serviço Windows. A instalação exige as permissões apropriadas do sistema. Execute service status para conferir o registro antes de iniciar.

```
nexusdb service install
nexusdb service status
nexusdb service start
nexusdb service stop
```

Ctrl+C encerra o modo console graciosamente, aguardando trabalho ativo e flush. Reinicie o processo depois de alterar pools ou limites lidos na inicialização. O serviço usa normalmente C:/ProgramData/NexusDB/config/nexusdb.env; confira os caminhos informados na instalação.

3 Conexão HTTP autenticação e respostas

3 1 Cabeçalhos comuns

Cabeçalho	Valor ou finalidade
Content-Type	application/json nas requisições JSON
X-User	Usuário do NexusDB
X-Pass	Senha da conta
X-Database	Banco lógico selecionado
X-Nexus-Execution-Id	Contexto opcional de execução nas rotas compatíveis

Use X-Database explicitamente. Um valor omitido em rotas legadas pode selecionar geo. A porta 7474 recebe a autenticação nativa. A chave de um endpoint do REST Gateway pertence ao processo e ao contrato do gateway.

A identidade da conta vem dos cabeçalhos autenticados. Os procedimentos que aceitam contexto não permitem assumir a identidade de outra pessoa pelo corpo JSON. O banco deve existir antes da inicialização de políticas ou da execução dos exemplos.

3 2 Cliente Python de referência

Este cliente usa somente a biblioteca padrão. Ele lê a senha do ambiente ou solicita sem eco, envia os cabeçalhos e verifica erros de statements. Crie o banco EXAMPLE_MANUAL pelo administrador conforme o capítulo 4.

```
import json
import os
from getpass import getpass
from urllib.request import Request, urlopen

URL = os.getenv("NEXUSDB_URL", "http://127.0.0.1:7474")
DATABASE = os.getenv("NEXUSDB_DATABASE", "EXAMPLE_MANUAL")
USER = os.getenv("NEXUSDB_USER", "admin")
PASSWORD = os.getenv("NEXUSDB_PASSWORD") or getpass("Senha: ")

def post(path, payload):
    request = Request(
        URL.rstrip("/") + path,
```

```
        data=json.dumps(payload).encode("utf-8"),
        headers={
            "Content-Type": "application/json",
            "X-User": USER,
            "X-Pass": PASSWORD,
            "X-Database": DATABASE,
        },
    )
    with urlopen(request, timeout=60) as response:
        value = json.load(response)
    if value.get("errors") or value.get("status") == "error":
        raise RuntimeError(value)
    return value

def query(text, parameters=None):
    return post("/db/data/cypher", {
        "include_statement_results": True,
        "statements": [{
            "query": text,
            "parameters": parameters or {},
        }],
    })

def procedure(name, request):
    return query("CALL " + name + "($request)",
                 {"request": request})
```

Os nomes de procedures desse cliente devem vir do código da aplicação. Parâmetros são valores estruturados. Uma resposta pode apresentar resultados externos agregados e os resultados completos de cada statement quando include_statement_results está ativo.

As amostras Python dos capítulos seguintes pressupõem esse cliente no mesmo arquivo ou importado. Para demonstrações, mantenha um módulo por laboratório e selecione explicitamente o banco correspondente.

3 3 Envelope e parâmetros

```
{
  "include_statement_results": true,
  "statements": [
    {
      "query": "CALL graph.project($request)",
      "parameters": {
        "request": {
          "projection": {},
          "limit": 10
        }
      }
    }
  ]
}
```

O objeto parameters do statement tem preferência sobre o fallback da requisição. A disponibilidade de parâmetros em um procedimento não representa suporte automático a expressões gerais em todas as cláusulas MATCH.

3 4 Falhas e semântica de lotes

Situação	Diagnóstico
HTTP 401	Confira usuário, senha e instância; observe eventual bloqueio por taxa

Situação	Diagnóstico
HTTP 403	Confira papel, grants e política Knowledge
HTTP 409	Examine conflito ou recurso não permitido no estado/modo atual
HTTP 429 ou 503	Confira limites de autenticação, fila e admissão
HTTP 200 com errors	Examine cada statement e os resultados parciais
status partial_error	Confira os itens concluídos antes de repetir uma escrita
statement_results	Leia o envelope de cada statement e seus campos adicionais

As garantias de lote variam por rota. O campo key correlaciona linhas e respostas; sua presença não cria idempotência de negócio. Uma nova tentativa após perda de resposta exige verificar o estado ou usar um mecanismo de idempotência suportado pela operação.

A mesma resposta de erro pode aparecer em errors para Cypher, em error textual para NPL e em um envelope estruturado para Agent. O cliente deve interpretar o contrato do endpoint utilizado.

Referências — HTTP e respostas, contexto de execução e limites de autenticação.

4 Bancos usuários e segurança

As operações nativas usam `X-User`, `X-Pass` e `X-Database`. Envie consultas a `POST /db/data/cypher` com o envelope abaixo; termine cada comando administrativo com ponto e vírgula.

```
{"statements":[{"query":"SHOW DATABASES"}]}
```

O catálogo distingue SUPERUSER global, atributo CREATEDB, propriedade do banco e concessões por banco. Uma conta com ADMIN em um banco não se torna SUPERUSER. Antes de criar usuários e concessões, autentique uma conta autorizada e selecione explicitamente o banco adequado.

```
CREATE USER operador WITH PASSWORD 'uma-senha-longa-e-exclusiva' LOGIN;
ALTER USER operador CREATEDB;
CREATE DATABASE EXAMPLE_VENDAS OWNER operador;
GRANT READ, WRITE ON DATABASE EXAMPLE_VENDAS TO operador;
SHOW GRANTS FOR operador;
REVOKE WRITE ON DATABASE EXAMPLE_VENDAS FROM operador;
ALTER DATABASE EXAMPLE_VENDAS OWNER TO admin;
DROP USER operador;
```

SUPERUSER controla usuários e atributos privilegiados. CREATEDB permite criar bancos para si próprio. OWNER possui acesso implícito total e pode transferir propriedade, remover o banco e conceder acessos. READ autoriza leitura; WRITE, escrita; ADMIN inclui administração daquele banco. `REVOKE` modifica concessões explícitas e não elimina acesso implícito de OWNER ou SUPERUSER. Para remover uma conta proprietária, transfira primeiro seus bancos. A troca da própria senha é permitida ao usuário autenticado e também desbloqueia sua conta.

Usuários, hashes, grants e propriedade ficam no catálogo local de cada nó. O HA não replica automaticamente esses comandos. Uma requisição com várias instruções administrativas não constitui transação coletiva: comandos anteriores podem ter concluído quando um posterior retorna erro, inclusive com `partial_error`. Planeje recuperação de uma execução parcial.

As rotas administrativas incluem `/db/admin/database/create`, `/db/admin/database/drop`, `/db/admin/grant`, `/db/admin/revoke` e `/db/admin/flush`. Remover o banco é diferente de truncar dados ou excluir um label. Não use comandos destrutivos como substitutos de uma seleção de banco ou de uma consulta de diagnóstico.

4 1 Política de senhas

Novas senhas usam Argon2id com salt aleatório. O mínimo padrão atual é 15 caracteres, limite de 1024 bytes e comparação com blocklist local. O documento SQL antigo menciona 12 caracteres; a implementação atual de segurança prevalece. Contas existentes continuam autenticando com sua senha anterior até alteração, sujeitas às políticas aplicáveis.

Variável	Padrão
NEXUSDB_ARGON2_MEMORY_KIB	19456
NEXUSDB_ARGON2_PASSES	2
NEXUSDB_ARGON2_LANES	1
NEXUSDB_AUTH_MAX_HASH_JOBS	2
NEXUSDB_AUTH_HASH_MEMORY_MB	256
NEXUSDB_AUTH_ACCOUNT_PER_MINUTE	300
NEXUSDB_AUTH_IP_PER_MINUTE	600

A fila de hashes admite até 32 esperas por até cinco segundos. Saturação retorna `AUTH_BUSY`; limitação de taxa pode retornar `AUTH_RATE_LIMIT`. A quota HTTP considera o peer TCP, não `X-Forwarded-For`. Cinco falhas bloqueiam a conta. Evite tentativas incorretas repetidas em scripts de diagnóstico.

4 2 Recuperação e segredos

Para recuperação, um administrador global emite um token privado; somente seu digest é persistido. O TTL aceito é 1–3600 segundos.

```
POST /db/admin/password/reset-token
{"username":"operador","ttl_seconds":900}

POST /db/auth/password/reset
{"reset_token":"TOKEN_PRIVADO","new_password":"nova-senha-longa-e-exclusiva"}

POST /db/admin/password/revoke-resets
{"username":"operador"}
```

A recuperação consome o token uma única vez, desbloqueia a conta e invalida outros tokens pendentes. Não há envio automático por e-mail. Uma falha de auditoria posterior à persistência pode produzir erro HTTP mesmo com a alteração concluída. Atualizações de credenciais invalidam sessões MCP anteriores; revogue também chaves independentes do gateway quando necessário.

DPAPI e pepper são opcionais. Prepare cofres sob a identidade que executa o serviço e mantenha o pepper fora do diretório de dados. Preserve chaves antigas enquanto contas ou backups dependerem delas; perda dessas chaves pode impedir autenticação. Não há fallback automático para texto puro quando a descriptografia falha.

Fontes: SQL de segurança, catálogo, política e limites, guia de segurança, usuários e grants.

4 3 Administração por HTTP

Com o cliente do capítulo 3 e uma conta administrativa, crie explicitamente o banco antes de usá-lo.

```
post("/db/admin/database/create",
     {"database": "EXAMPLE_MANUAL"})
post("/db/meta/databases", {})
```

POST /db/admin/database/drop recebe database. É uma operação destrutiva sobre o banco informado; não é necessária para selecionar um banco existente.

```
post("/db/admin/user/create", {
    "username": "leitor_manual",
    "password": "Substitua-por-senha-unica-2026!",
    "role": "user",
    "expires_days": 0,
})
post("/db/admin/grant", {
    "username": "leitor_manual",
    "database": "EXAMPLE_MANUAL",
    "access": "READ",
})
```

O corpo de POST /db/auth/password/change recebe new_password e usa a credencial atual do próprio usuário. Certas rotas /db/admin exigem o papel global admin, além das permissões por banco. A referência de cada operação define essa exigência.

5 Dados consultas e índices

O endpoint `POST /db/data/cypher` reúne consultas, atualizações e procedimentos nativos. Envie `X-User`, `X-Pass` e `X-Database`, escolhendo um banco ao qual o usuário tenha acesso. A autorização é verificada antes da execução: consultas exigem leitura e mutações exigem escrita. Os exemplos usam dados de demonstração; IDs de nós devem ser substituídos pelos IDs retornados na instalação utilizada.

```
{"statements":[{"query":"MATCH (n:Person) RETURN n LIMIT 10"}]}
```

Cada item de `statements` contém `query` e pode conter `parameters`. Os parâmetros locais prevalecem sobre `parameters` na raiz da requisição e são valores estruturados, não trechos de código. Para procedimentos que recebem mapas, este formato evita montar strings manualmente:

```
{"include_statement_results":true,"statements":[{"query":"CALL
graph.project($request)","parameters":{"request":{"projection":{"layers":["care"]},"limit":20}}}]}
```

A resposta contém `status`, `database`, `processados`, `resultados`, `errors` e `execution`. HTTP 200 não basta para concluir sucesso: verifique `errors` e `status`, que pode ser `partial_error`. Statements são processados individualmente; uma falha posterior não desfaz automaticamente um statement anterior. `include_statement_results:true` acrescenta os envelopes completos dos procedimentos em `statement_results`, úteis para limites, snapshots e metadados.

5 1 Subconjunto de consultas

O MATCH nativo representa um padrão de nó ou uma única relação. Ele não implementa cadeias gerais de vários saltos. Use consultas pequenas e confira a forma do retorno:

```
MATCH (n:Person) RETURN n LIMIT 10
MATCH (a:Person)-[r:KNOWS]->(b:Person) RETURN a,r,b LIMIT 10
MATCH (n:Person) WHERE n.age=31 RETURN n
```

A existência de palavras ou funções no ecossistema Cypher não garante sua execução aqui. `WITH`, `UNWIND`, `OPTIONAL MATCH`, `MERGE`, `UNION`, `DISTINCT`, ordenação geral com `ORDER BY`, `SKIP`, padrões múltiplos e agrupamento não constituem recursos disponíveis do subconjunto nativo. Não há declaração de conformidade completa openCypher nem aprovação pela suíte TCK. Consulte compatibilidade openCypher.

COUNT existe no adaptador de padrão de nó único:

```
MATCH (n:Person) RETURN count(n)
MATCH (n:Person) RETURN count(*)
MATCH (n:Person) RETURN count(n.age)
```

`count(n.age)` exclui propriedades ausentes e valores null. Não extrapole para contagem de relações, agrupamento por coluna ou agregações SUM/AVG. Misturar contagem com uma coluna comum é rejeitado como agrupamento não suportado. O projetor legado de RETURN não calcula expressões gerais; uma expressão desconhecida pode produzir a linha original por compatibilidade, e não a agregação pretendida.

Existe uma extensão antiga de caminhos variáveis:

```
MATCH p=(a)-[*1..3]->(b) RETURN p LIMIT 10
```

Essa rota enumera caminhos simples dirigidos, globalmente, a partir de todos os nós. Ela extrai a faixa de profundidade e o limite, mas **ignora WHERE, labels, tipos de relação e bindings do padrão**. Não acrescente filtros esperando restringir a exploração. A seta textual também não é uma instrução geral de direção: o algoritmo utiliza adjacências de saída. O parser MATCH nativo rejeita caminhos variáveis; a extensão funciona porque é interceptada antes dele. Não use esse mecanismo para regras de acesso, filtros de negócio ou

enumeração extensa. Os padrões são profundidade 1 a 3 e 100 caminhos quando faltam valores reconhecidos. Prefira os procedimentos explícitos do capítulo seguinte.

5 2 Criação e atualização

Execute os exemplos em statements separados:

```
CREATE (n:Person {name:'Ana',age:31}) RETURN id(n) AS id
MATCH (n:Person {name:'Ana'}) WHERE n.age=31 SET n.age=32 RETURN n.age
MATCH (n:Person) WHERE id(n)=1 REMOVE n.age
MATCH (n:Person) WHERE n.age IS NULL RETURN count(n)
MATCH (n:Person) WHERE id(n)=1 DELETE n
MATCH (n:Person) WHERE id(n)=1 DETACH DELETE n
```

O adaptador valida o subconjunto antes de escrever e persiste cada statement numa transação SQLite. Os índices e caches são atualizados depois da persistência. DELETE de nó conectado falha antes de alterar dados; DETACH DELETE remove também as relações incidentes. IDs restantes não são renumerados. SET com null remove a propriedade. Igualdade com null não seleciona linhas; use IS NULL ou IS NOT NULL.

LIMIT restringe a resposta, não a quantidade de nós alterados. A seleção do adaptador respeita `NEXUSDB_TEMPORAL_RESULT_LIMIT`, padrão 1000, e exceder esse limite falha antes da mutação. Atualizações desse adaptador não executam em HA, pois não existe replicação de seus statements. O CREATE relacional legado com IDs dos endpoints mantém contrato próprio; não confunda com criação arbitrária de padrões Cypher.

5 3 Escalares e índices

RETURN constante exige AS em cada coluna. São aceitos literais, mapas/listas de literais e funções aninhadas do subconjunto:

```
RETURN size('é😀') AS caracteres, toInteger('12') AS inteiro
RETURN substring('abcdef',2,3) AS trecho, coalesce(null,'padrão') AS valor
```

Entre as funções disponíveis estão conversões, size, coalesce, operações de caixa e trim, reverse, head/last/tail, keys, split, substring, left/right e abs. A disponibilidade em RETURN constante não significa disponibilidade dentro de MATCH/WHERE. Aridade, tipos, tamanho, profundidade e memória são validados.

Os índices de propriedade do executor e os índices vetoriais são recursos distintos. A criação de índice de propriedade segue `CREATE INDEX ON :Person(name)`; use a grafia de label e campo adotada pelo conjunto de dados. Índices vetoriais têm definição, dimensão e métrica explícitas no capítulo 8. Os índices atendem aos caminhos de consulta implementados no NexusDB.

5 4 Operações HTTP de dados

POST /db/data/node/create recebe label e properties. A criação em lote recebe rows; cada linha tem key, label e properties. O resultado devolve IDs e keys, que permitem montar as relações com from, to, type e properties.

```
post("/db/data/node/search",
     {"label": "Produto", "limit": 10})
post("/db/data/node/update", {
    "id": ID_RETORNADO,
    "properties": {"estoque": 30},
    "merge": True,
})
post("/db/index/create",
     {"label": "Produto", "field": "sku"})
```

ID_RETORNADO representa uma variável da aplicação com o ID real. merge true combina propriedades. node/detach remove relações incidentes; node/delete e detach delete têm seus próprios contratos de exclusão. O laboratório do capítulo 16 demonstra uma criação e leitura completas.

Referências de implementação — parser, atualizações, retorno e rotas.

6 Algoritmos planejamento e computação

Os procedimentos abaixo são operações nativas sobre o grafo selecionado. Leitura autorizada continua obrigatória. Uma relação estrutural não estabelece significado de negócio: um caminho entre entidades heterogêneas pode existir sem representar uma rota válida para a aplicação.

6 1 Catálogo de procedimentos de grafo

Chamada	Uso e argumentos
`CALL algo.degree() LIMIT 20`	Grau, ordenado; retorno padrão até 100 nós
`CALL algo.pageRank(20) LIMIT 20`	Iterações posicionais; damping 0,85
`CALL algo.connectedComponents()`	Componentes conectados
`CALL algo.shortestPath(1,10)`	Caminho por quantidade de saltos, não dirigido
`CALL algo.dijkstra(1,10,'peso')`	Caminho ponderado não dirigido; campo padrão peso
`CALL algo.betweenness(50) LIMIT 20`	Amostragem posicional e limite de retorno
`CALL algo.community()`	Label propagation, 20 iterações
`CALL graph.community()`	Mesmo alias de comunidade
`CALL graph.labelPropagation(20)`	Número de iterações
`CALL graph.louvain(20)`	Número de iterações
`CALL graph.bfs(1,3)`	Origem e profundidade máxima
`CALL graph.dfs(1,3)`	Origem e profundidade máxima
`CALL graph.expand(1,3) LIMIT 100`	Expansão BFS
`CALL graph.statistics()`	Estatísticas do grafo
`CALL graph.degreeDistribution()`	Distribuição de graus
`CALL graph.density()`	Densidade
`CALL graph.giantComponent()`	Componente gigante
`CALL graph.randomModel()`	Estimativa do modelo aleatório
`CALL graph.phaseTransition()`	Alias da mesma estimativa
`CALL graph.diameterApprox(50)`	Diâmetro aproximado por amostragem

Chamada	Uso e argumentos
`CALL graph.centrality()`	Alias de degree
`CALL graph.influence()`	Alias de PageRank, 20 iterações
`CALL graphRAG.context(1,2)`	Contexto estrutural por origem/profundidade
`CALL graphRAG.prompt(1,2)`	Materialização de contexto em formato de prompt
`CALL graph.project({projection: {},limit:20})`	Materialização de projeção, detalhada adiante

BFS, DFS e expand operam de forma não dirigida, incluem a origem e usam profundidade padrão 2. O limite padrão de retorno é 100. Nesse caminho, LIMIT reduz a materialização após a travessia; não é orçamento de nós explorados. shortestPath e Dijkstra dessas chamadas também são não dirigidos. O peso de Dijkstra é uma propriedade de custo; não significa metros sem uma definição explícita no conjunto de dados.

Degree e PageRank usam snapshots versionados. A ordenação é decrescente pela medida, com desempate pelo ID. O processamento pesado ocorre após a captura do snapshot; se as propriedades atuais mudarem antes de materializar a resposta, a chamada pode devolver `COMPUTE_SNAPSHOT_CHANGED`, solicitando nova consulta. Alguns procedimentos antigos extraem argumentos de forma permissiva: mantenha as assinaturas documentadas em vez de supor validação Cypher geral.

6 2 Planejamento, cache e telemetria

```
EXPLAIN MATCH (n:Person) RETURN n LIMIT 10
PROFILE MATCH (n:Person) RETURN n LIMIT 10
CALL db.query.stats()
CALL algo.tuningStats()
```

O planejador GRASP atua sobre o padrão suportado de um nó ou uma relação. O plano físico informa scan, filtro, expansão e projeção; a execução de MATCH informa nodes_visited e edges_visited. PROFILE mantém a função de explicar estimativas e não representa uma execução instrumentada completa.

O cache conserva identidade por texto exato e banco. Espaços e caixa dentro dos literais são significativos. Existem caches de AST e plano físico; não existe cache de resultados. Uma consulta repetida lê o estado atual. Mutações, índices, restauração e recriação invalidam as revisões relevantes. `NEXUSDB_QUERY_CACHE=false` desativa AST e plano.

Cada cache tem até 128 entradas e estimativa de 4 MiB por banco/32 MiB global. Esses números não limitam o RSS total. Contadores e eventos recentes de `db.query.stats` são voláteis. `NEXUSDB_QUERY_TELEMETRY_FILE` habilita JSONL assíncrono: fila de 1024 eventos, rotação de 8 MiB, com descarte contado em saturação. Não é auditoria durável. Veja runtime de consultas.

6 3 Backends e calibração

Configuração	Padrão
`NEXUSDB_COMPUTE_BACKEND`	AUTO
`NEXUSDB_COMPUTE_THREADS`	Paralelismo disponível

Configuração	Padrão
NEXUSDB_COMPUTE_MEMORY_MB	256
NEXUSDB_COMPUTE_MAX_JOBS	2
NEXUSDB_COMPUTE_TIMEOUT_MS	30000

Os valores de backend são AUTO, CPU_SERIAL, CPU_PARALLEL e CUDA. CUDA é opcional e exige compilação com feature cuda, biblioteca compatível e driver NVIDIA. Nem toda operação possui kernel CUDA. Verifique `requested_backend`, `selected_backend`, `reason`, tempos, versão e fallback na telemetria; igualdade de resultados não comprova uso de GPU. Timeout e cancelamento são cooperativos, podendo ser observados apenas entre etapas.

Com autorização ADMIN adicional:

```
CALL algo.calibrate('degree',1,5)
CALL algo.calibrate('pageRank',100,5)
```

As repetições vão de 5 a 15 e PageRank aceita até 1000 iterações nessa interface. A calibração compara resultados e tempos; só substitui serial quando a redução de mediana supera 15%. Enquanto calibra, outros jobs de compute recebem COMPUTE_BUSY. Perfis exatos dependem do snapshot e vivem no processo; buckets empíricos também podem persistir em `data/cache/tuning.json`. `algo.tuningStats()` expõe o estado de tuning.

SpMV reutiliza o pool e orçamento compartilhados:

```
CALL graph.spmv({values:[1,2,3],projection:{weight_property:'weight'}}) YIELD node_ids,values RETURN
node_ids,values
CALL graph.spmvBoolean({values:[false,true,false]}) YIELD values RETURN values
```

O vetor acompanha a ordem crescente dos IDs da projeção; ajuste seu tamanho. A operação numérica calcula soma dos pesos de saída multiplicados pelas entradas; a booleana usa OR/AND. Peso ausente, inválido ou negativo segue fallback nativo para 1. CUDA explícito é rejeitado para essas operações.

Referências de implementação — algoritmos e dispatch, travessias, compute e planejador.

7 Histórico projeções e armazenamento em disco

7 1 Validade e filtros

Uma projeção seleciona topologia e propriedades persistidas para construir CSR derivado e reutilizável. IDs e arestas paralelas são preservados.

```
CALL graph.project({projection:{temporal:{kind:'at',timestamp:15},layers:['care']},limit:100})
CALL graph.projectDegree({projection:{temporal:
{kind:'window',start:10,end:30,mode:'any'}},limit:20})
CALL graph.projectPageRank({projection:{layers:['care']},iterations:20,damping:0.85,limit:20})
```

Campo de projection	Contrato
system_at	Instante confirmado do histórico, Unix ms UTC
temporal	Validade dos relacionamentos: at ou window
node_temporal	Validade dos nós

Campo de projection	Contrato
layers	Lista de camadas, sensível à caixa
node_filters	Filtros de propriedades de nós
relationship_filters	Filtros de relações
weight_property	Propriedade de peso
spatial	Filtro espacial de nós
geographic_weights	Pesos geográficos nas operações compatíveis

Filtros de propriedade usam `{field:'risk',operator:'ge',value:3}`. Operadores: eq, lt, le, gt, ge; ordenação exige números, sem conversão de strings. As conjunções são AND. Nós filtrados formam subgrafo induzido: ambas as pontas passam. Propriedades são de primeiro nível; ausência difere de null.

Validade utiliza inteiros em milissegundos, sem interpretar datas textuais. Uma relação pode guardar `valid_from`/`valid_to`, ou exclusivamente `temporal_intervals:[{valid_from:10,valid_to:20}]`. Os intervalos são semiabertos; término ausente/null significa aberto. Lista vazia nunca está ativa; entidade sem metadados temporais permanece atemporal. A janela any seleciona alguma atividade e all exige cobertura contínua, considerando a união dos intervalos.

project retorna IDs e contagens, não exporta todas as propriedades. Degree/PageRank projetados retornam medidas ordenadas. Padrões: limit 100, iterations 20, damping 0,85; limite/iterações positivos e damping entre 0 e 1. PageRank continua estrutural e não ponderado. O cache é versionado, com até quatro entradas por banco e orçamento compartilhado estimado. Materialização captura estado consistente sob lock; construção CSR e ordenação não são interrompidas no meio. Consulte projeções temporais.

7 2 Histórico transacional

O histórico é persistente por banco e desativado por padrão. ADMIN pode habilitá-lo durante a criação ou posteriormente:

```
POST /db/admin/database/temporal-history
{"database":"ensaio","temporal_history":true}
```

A baseline é atômica. Repetir true é idempotente; depois de iniciado, não se permite desligar e criar lacunas. A variável global antiga NEXUSDB_TEMPORAL_HISTORY é ignorada. Não existe recuperação de versões anteriores à ativação nem expurgo automático.

```
CALL graph.historyStatus({})
CALL graph.historyEvents({entity:'nodes',id:1,after_seq:0,limit:100})
CALL graph.project({projection:{system_at:1791750000000},limit:100})
```

Substitua system_at por um instante coberto pelo banco, obtido de `stamp.recorded_at` em historyStatus. Instantes anteriores à baseline ou posteriores à cobertura são rejeitados. historyEvents pagina em seq crescente; use o último seq como after_seq. A baseline não aparece nessa lista. O histórico desabilitado tem status consultável, mas consultas históricas falham.

system_at representa o que era conhecido na confirmação; temporal representa validade de negócio naquele estado. A combinação permite correções retroativas sem modificar o conhecimento anteriormente registrado. O watermark é monotônico por banco, não um relógio distribuído entre réplicas.

```
CALL graph.earliestArrival({source:1,target:3,departure:1790812800000,max_depth:8})
CALL
graph.temporalBfs({source:1,target:3,departure:1790812800000,deadline:1790815200000,max_depth:8,max_
states:10000})
```

Os caminhos são dirigidos e permitem esperar abertura da relação. duration_ms é inteiro não negativo, padrão zero. A travessia começa antes de valid_to e pode chegar exatamente no término. earliestArrival minimiza chegada, depois saltos; temporalBfs minimiza saltos, depois chegada. Não aceitam projection.temporal: use departure/deadline. Aceitam system_at, camadas e filtros. Os defaults da requisição são max_depth 32/max_states 10000; os tetos configuráveis são distintos. Exceder estados causa erro, sem aproximação silenciosa.

7 3 Snapshots, partições e Arrow

```
CALL graph.diskBuild({snapshot_id:'ensaio',projection:{layers:['care']}})
CALL graph.diskBfs({snapshot_id:'ensaio',source:1,max_depth:4,max_nodes:100})
CALL graph.partitionBuild({snapshot_id:'partes',partition_span:2048,projection:{layers:['care']}})
CALL graph.partitionBfs({snapshot_id:'partes',source:1,max_depth:8,max_visited:100000,limit:100})
CALL graph.partitionNode({snapshot_id:'partes',id:1})
CALL graph.historyCheckpoint({})
```

IDs de snapshot são seguros: letras ASCII, números, hífen/sublinhado, até 64 caracteres. Snapshots são imutáveis, não sobrescritos nem atualizados por novas escritas. diskBuild monolítico ainda materializa CSR em RAM; diskBfs valida checksum e lê páginas de 64 KiB. partitionBuild constrói por streaming SQL sem CSR integral; partition_span padrão 65536. partitionBfs é dirigida, com max_depth 2, max_visited 100000 e limit 100 por padrão. max_visited restringe travessia, limit somente resposta; truncated/results_truncated distinguem os cortes. partitionNode lê propriedades sob demanda.

Checkpoints históricos guardam reconstruções SQLite e não removem eventos. Aceitam system_at opcional e reutilizam versão já existente. Backup SQLite preserva histórico; snapshots JSON/install_snapshot iniciam nova baseline. Recrie derivados após restauração quando necessário.

```
CALL
graph.exportArrow({export_id:'people',entity:'nodes',batch_rows:512,batch_bytes:1048576,projection:
{},columns:[{name:'age',field:'age',data_type:'int64'}]})
```

Arrow recebe entity nodes/relationships, columns com name/field/data_type e projection. Defaults: 1024 linhas e 1MiB por lote. Tipos boolean, int64, uint64, float64, utf8 e json; incompatibilidade aborta sem coerção. Colunas auxiliares __present distinguem ausência de null; properties_json conserva mapa completo. O arquivo é Arrow IPC stream, não Parquet. Criar export exige escrita; download por `POST /db/data/arrow/download` exige leitura no banco. Veja armazenamento particionado para corpo de download e importação offline.

Construções, checkpoint e exportação exigem WRITE; consultas exigem READ. Temporal/history/disk rejeitam CUDA explícito. Modo disk-only admite somente operações explicitamente compatíveis: não oferece MATCH arbitrário, PageRank, caminhos temporais ou GraphRAG em memória. Não há atualização incremental das partições. Tetos padrão: resultado 1000, profundidade 64, estados 100000, arestas GraphRAG 100000, disco 1024 MiB; configuráveis por NEXUSDB_TEMPORAL_RESULT_LIMIT/MAX_DEPTH/MAX_STATES/MAX_EDGES/DISK_LIMIT_MB. Consulte motor temporal.

Referências de implementação — projeções, histórico e disco, caminhos temporais e histórico.

8 Vetores GraphRAG e dados espaciais

8 1 HNSW e evidências de grafo

Os embeddings são fornecidos pelo cliente; o banco não os gera implicitamente. Execute separadamente:

```
CREATE (:Chunk {nome:'Rust',embedding:[1.0,0.0,0.0,0.0]})
CALL vector.createIndex('chunk_embeddings','Chunk','embedding',4,'cosine')
CALL vector.listIndexes()
CALL vector.search('chunk_embeddings',[1.0,0.0,0.0,0.0],10,64)
CALL graphRAG.retrieve('chunk_embeddings',[1.0,0.0,0.0,0.0],10,64,2,100)
CALL vector.dropIndex('chunk_embeddings')
```

Parâmetro	Contrato
createIndex	nome, label, propriedade, dimensão, métrica
search	índice, vetor, k=10, ef=64
retrieve	argumentos de search, depth=2, limit=100
Dimensão	1 a 65536, igual no índice e consulta
Métrica	cosine ou euclidean
k / ef	1..1000 / 1..4096; ef efetivo pelo menos k
GraphRAG	depth 0..8, até 1000 nós e 10000 arestas examinadas

Valores devem ser finitos e representáveis em f32; cosseno rejeita vetor zero. Labels/propriedades usam grafia exata; índice é insensível à caixa. Há até 32 definições por banco e orçamento global derivado estimado 256 MiB, separado do RSS e dos dados principais.

HNSW é aproximado e usa CPU. ef maior aumenta trabalho e pode melhorar recall, sem garantia universal. Definições persistem em vector_indexes.sqlite; o grafo HNSW é reconstruído após revisão/reinício, sob lock. Não há manutenção incremental. Embedding inválido impede reconstrução e busca, sem reutilizar índice antigo silenciosamente. Listar/remover continua possível. CREATE/DROP exigem WRITE e líder/SINGLE; definições seguem snapshots, sem replicação incremental dedicada do DDL vetorial.

retrieve expande relações de saída a partir das sementes e devolve evidências com limites/truncated. Similaridade vetorial não prova relação estrutural, causalidade ou confiança. O banco não chama LLM nem produz resposta final automaticamente. Mais detalhes em HNSW e GraphRAG.

Temporal GraphRAG aceita sementes explícitas ou vetor, exclusivamente:

```
CALL graphRAG.temporalRetrieve({seeds:[1],depth:2,max_nodes:100})
CALL graphRAG.temporalRetrieve({vector:
[1,0,0,0],seed_label:'Chunk',embedding_property:'embedding',k:1,depth:2,max_nodes:100})
```

A busca histórica é exata por cosseno e não usa HNSW corrente. Dimensões incompatíveis causam erro. projection pode conter system_at e filtros; a expansão retorna propriedades/versionamento e caminhos de evidência. Uma janela estrutural não estabelece caminho cronológico.

8 2 Importação e consulta espacial

```
CALL spatial.importGeoJSON({label:'Place',collection:{type:'FeatureCollection',features:
[{type:'Feature',id:'p1',properties:{name:'A'},geometry:{type:'Point',coordinates:[-46.63,-
23.55]}}]}})
CALL spatial.search({label:'Place',projection:{spatial:
{kind:'radius',longitude:-46.63,latitude:-23.55,radius_m:3000}},limit:100})
CALL spatial.exportGeoJSON({projection:{}})
CALL spatial.shortestPath({start:1,end:4,directed:true,projection:{layers:['care']}})
```

Importação exige WRITE e valida coleção antes de commit; demais procedimentos exigem READ. IDs externos/propriedades são preservados. projection combina filtros espaciais, validade de nós/relações, camadas e propriedades por AND. Search/export rejeitam filtros de relações, evitando opções ignoradas. O limite espacial padrão é 1000, configurável por NEXUSDB_SPATIAL_RESULT_LIMIT; excesso falha em vez de truncar silenciosamente.

Os filtros espaciais escolhem nós Point: bbox pode cruzar antimeridiano; radius usa metros; within recebe Polygon WGS84. R-tree é pré-seleção do estado corrente; histórico e disk-only usam fonte SQLite consistente. Search/export são compatíveis com disk-only, mas shortestPath não é.

```
RETURN spatial.point({longitude:-46.63,latitude:-23.55,srid:4326}) AS p
RETURN
spatial.distance(spatial.point({longitude:0,latitude:0,srid:4326}),spatial.point({longitude:1,latitu
de:0,srid:4326})) AS metros
```

Escalares adicionais: spatial.within, intersects, length e transform. point recebe mapa, não dois argumentos posicionais. São funções do RETURN constante, sem promessa de uso geral em MATCH/WHERE. SRIDs 4326 e 3857 são aceitos; distância geográfica utiliza esfera de raio 6371008,8 m, não geodésia elipsoidal. Predicados exigem mesmo CRS; within exclui borda e intersects inclui. length aceita LineString para cálculo; armazenamento canônico suporta Point e Polygon de anel único.

Legacy ST_DWithin continua euclidiano nas unidades originais: graus não viram metros. Caminho geográfico utiliza Haversine em metros sobre relações existentes; não representa automaticamente rede viária. Não misture distance_m com custos/tempos. Pesos geográficos em disco são rejeitados; use propriedade pré-calculada compatível.

Polígonos com buracos, auto-interseção, anéis abertos/degenerados, cruzamento do antimeridiano, mais de 1024 vértices ou CRS não suportado são rejeitados. Exportação não reprojeta Polygon 3857. O conversor GeoParquet offline exige feature spatial-geoparquet, aceita WKB Point 2D/CRS84 implícito, IDs int64 não negativos únicos e destino novo. Não transforma Parquet em armazenamento histórico transacional. Consulte integração espacial.

8 3 Verificação reproduzível

Estes comandos correspondem a targets presentes no repositório; sua inclusão não registra execução nesta revisão:

```
cargo test --offline --lib
cargo test --offline --test query_runtime --test grasp_runtime --test graph_runtime
cargo test --offline --test temporal_history_runtime --test temporal_projection_runtime
cargo test --offline --test vector_runtime
cargo test --features spatial-geoparquet --test spatial_runtime --test geoparquet_runtime
cargo test --test compute_config
```

O teste csr_runtime_integration pertence ao target graph_runtime; não existe target separado com esse nome. Os testes de integração usam isolamento próprio. Para reproduções manuais, utilize data root dedicado, credencial de teste e banco novo; exemplos de CREATE/importação alteram dados e não devem ser tratados como consultas somente leitura.

9 Conhecimento memórias e descobertas

O módulo de conhecimento registra memórias com origem, validade e controle de leitura; recupera evidências e acompanha descobertas verificáveis. Ele reutiliza armazenamento, histórico, vetores e recursos nativos. Os procedimentos não constituem verificação factual independente nem oferecem compatibilidade integral com RDF ou openCypher. Implementação: rotas, dispatcher, política.

9 1 Preparação e primeira memória

Compile com `knowledge`; as features `phronesis` e `datalog` também habilitam conhecimento. Esses recursos são opcionais e desabilitados por padrão. Para o conjunto deste capítulo:

```
cargo build --features phronesis,datalog,embedded,spatial-geoparquet --bins
```

Use `POST /db/data/cypher` com `X-User`, `X-Pass` e `X-Database`. Uma chave do REST Gateway não substitui essas credenciais. Antes das operações knowledge/memory/phronesis, um administrador deve executar, no database escolhido:

```
{"statements":[{"query":"CALL knowledge.initialize({})"}]}
```

`policy::initialize` exige ADMIN e objeto vazio, habilita histórico e persiste a política scoped naquele database. Essa operação altera o contrato de acesso: usuários comuns passam pelos procedimentos com escopo; interfaces brutas não devem contornar a política. ADMIN continua operador confiável com acesso bruto. Não execute a ativação em um database existente sem considerar essa mudança de acesso. Escritas knowledge em HA e armazenamento disk-only não estão disponíveis nesta integração.

Para registrar uma preferência de compras com origem explícita:

```
{
  "include_statement_results":true,
  "statements":[{
    "query":"CALL memory.remember($request)",
    "parameters":{"request":{
      "content":"A equipe prioriza qualidade e prazo na seleção de fornecedores.",
      "origin":{"kind":"document","id":"politica-compras","version":"v1"},
      "kind":"preference",
      "idempotency_key":"compras-preferencia-v1"
    }}
  }]
}
```

O ator vem da autenticação. A escrita exige GRAFO_WRITE; as leituras exigem GRAFO_READ separadamente. WRITE não concede READ automaticamente. `include_statement_results` conserva os metadados individuais dos CALLs. Parâmetros evitam incorporar conteúdo de documentos diretamente à sintaxe da consulta.

`memory::Remember` exige `content` e `origin`; origem possui `kind`, `id` e `version`, com kind user/document/tool/llm. Os tipos de memória são episodic, preference, hypothesis, knowledge e case. Campos adicionais incluem `context`, `valid_from`, `valid_to`, `readers`, `evidence_ids`, `embedding`, `claim`, `geometry` e `case`. O conteúdo é limitado a 65.536 bytes; leitores não aceitam `*`. Embeddings declarados incluem model_id, values e metric; não são gerados pelo procedimento. Fonte: memory::Remember e validate.

9 2 Catálogo de procedimentos

Cada procedimento abaixo recebe exatamente um objeto. O nome é normalizado para minúsculas. A tabela apresenta todas as entradas executáveis desta família, incluindo aliases.

Procedimento	Entrada e comportamento
`knowledge.initialize`	`{}`; ADMIN; ativa política e histórico
`memory.remember`, `knowledge.remember`	Objeto Remember; publica registro durável
`knowledge.assertClaim`	Remember com `claim:{subject,predicate,object,functional?}`
`memory.supersede`	`node_id,expected_revision,replacement`; substituição com revisão otimista
`memory.retract`, `memory.forget`, `knowledge.retract`	`node_id,expected_revision,reason,context?`; retirada lógica
`memory.promote`	Mesmo formato de retirada; exige evidências disponíveis
`knowledge.search`	Busca lexical BM25
`knowledge.hybridSearch`, `memory.recall`	Recuperação híbrida lexical/vetorial/estrutural
`knowledge.contextPack`, `memory.contextPack`	Evidências citadas dentro de orçamento de caracteres
`memory.reflect`, `memory.consolidate`	Propostas de revisão; `persisted:false`
`knowledge.verifyClaim`	`subject,predicate,object,context?`; confronto com alegações disponíveis
`graph.ppr`	Seeds e parâmetros PPR sobre snapshot autorizado
`graph.schema`, `graph.health`	Contexto opcional; esquema observado e inspeção
`graph.propertyValues`	`label,key,limit?,context?`; valores observados
`graph.changeFeed`	`after_sequence?,epoch?,limit?`; ADMIN; commits confirmados
`graph.window`	`rows,partition_key,order_key,value_key,row_key`; agregações e posições JSON

Fontes: routes::is_write/handle, runtime::execute, retrieval::Search/search.

Exemplo de recuperação:

```
{"statements":[{"query":"CALL knowledge.search({query:'fornecedores',top_k:5})"}]}
```

Search aceita context, query, top_k, method, rrf_k, lexical_weight, vector_weight, graph_weight, embedding, seeds, alpha, tolerance, max_iterations, max_chars e kind. Defaults: top_k=10, method=rrf, rrf_k=60, alpha=0.15, tolerance=1e-8, max_iterations=100 e max_chars=8192. Modelos vetoriais diferentes não são comparados; dimensões incompatíveis são recusadas. BM25 normaliza caixa e acentos latinos, sem stemming. RRF/DBSF produzem sinais de ranking, não probabilidades.

9 3 Escopo, validade e erros

`context` aceita `scope` user/session/team/organization/project, `scope_id` e `projection`. O padrão é o usuário autenticado; escopos compartilhados exigem ID explícito. Proprietário e readers controlam leitura. Não existe diretório externo de equipes ou ABAC jurídico completo. ACL é aplicada antes do corpus, estatísticas e ranking. IDs e metadados de commits são globais no database; para isolamento desses metadados, use databases separados. Fonte: snapshot::Scope e capture.

`projection.system_at` seleciona o conhecimento registrado em um instante; `projection.node_temporal` seleciona validade de negócio. Intervalos são `[início,fim)`; sem projeção temporal, a validade usa o instante atual. Consultar história não ignora a ACL atual. Retração de fonte impede recuperar derivados na visão corrente e pode redigir a explicação de decisões dependentes.

Retração não elimina fisicamente dados. Supersede preserva trechos temporais fora do intervalo substituído. Promoção mantém o grau asserted: não certifica verdade. Context packs contêm dados não confiáveis e citações, sem resposta gerada; max_chars não é orçamento de tokens. Sem evidência, indicam insuficiência.

Erros representativos são `KNOWLEDGE_FEATURE_DISABLED`, `KNOWLEDGE_POLICY_REQUIRED`, `KNOWLEDGE_ACCESS_DENIED`, `KNOWLEDGE_INVALID_REQUEST`, `KNOWLEDGE_REVISION_CONFLICT` e `KNOWLEDGE_REFERENCE_UNAVAILABLE`. Um conflito de revisão exige reler o registro antes de decidir uma nova escrita; não faça retry automático cego. O feed exige cursor da época correta e recusa cursor futuro. Limites padrão incluem cem mil nós por snapshot e mil resultados; reservas são estimativas, não isolamento físico de memória.

9 4 Descobertas e validações reproduzíveis

A implementação atual inclui sete procedimentos adicionais. Os registros são imutáveis e validações são novos eventos, não edição de resultados antigos. Fonte completa: discovery::execute/write e structs de entrada.

Procedimento	Campos obrigatórios, além de contexto opcional
`knowledge.discovery.task`	`idempotency_key,task_key,version,objective,verifier`; ADMIN
`knowledge.discovery.line`	`idempotency_key,task_id,approach`
`knowledge.discovery.submit`	`idempotency_key,line_id,kind,content,artifact_version,conditions,reproduction`
`knowledge.discovery.validate`	`idempotency_key,node_id`; attestation conforme verificador
`knowledge.discovery.adopt`	`idempotency_key,line_id,validation_id,variation`
`knowledge.discovery.feed`	`task_id,version`; paginação opcional
`knowledge.discovery.diversity`	`task_id,version`; consulta de diversidade das linhas

Exemplo de argumento para task:

```
{
  "idempotency_key":"compras-task-v1",
  "task_key":"comparacao-fornecedores",
  "version":"v1",
  "objective":"Reproduzir uma transformação JSON de referência",
  "verifier":{"kind":"json_pipeline_v1","cases":[{"input":1,"expected":1}]}
}
```

Envie-o como `$request` em `CALL knowledge.discovery.task($request)`, no envelope parametrizado anterior. Os IDs retornados alimentam line, submit e validate. Uma abordagem normalizada já reservada é recusada. Submit permite hypothesis/experiment/result/evidence, artifact opcional, evidências e links SUPORTA/REFUTA/DERIVA_DE; somente experiment/result são candidatos a validação.

`json_pipeline_v1` executa testes do contrato no servidor, usando operações JSON limitadas. `external_metrics_v1` recebe métricas declaradas por validadores autorizados: não executa o experimento externo. A qualificação do resultado distingue replay nativo de declaração externa. Adopt exige validação aprovada, sem validação posterior do mesmo candidato, referências autorizadas e mesma tarefa. O resultado vale para artefato, versão, contrato e condições registrados; não demonstra superioridade universal. Feed/diversity aceitam after_id, epoch, limit e max_bytes. Erros incluem `DISCOVERY_TASK_REQUIRES_ADMIN`, `DISCOVERY_APPROACH_ALREADY_CLAIMED`, `DISCOVERY_LINE_OWNER_REQUIRED`, `DISCOVERY_ATTESTATION_REQUIRED` e `DISCOVERY_NEWER_VALIDATION_EXISTS`.

10 Phronesis e Datalog

10 1 Avaliação de fornecedores

Phronesis oferece avaliação multicritério determinística e decisões auditáveis. Exige feature phronesis e política knowledge inicializada. Este exemplo compara qualidade e prazo, com pesos 0,6/0,4; C é inelegível por uma restrição rígida:

```
{
  "include_statement_results":true,
  "statements":[{
    "query":"CALL phronesis.evaluate($request)",
    "parameters":{"request":{"evaluation":{
      "weights":{"qualidade":0.6,"prazo":0.4},
      "alternatives":[
        {"id":"A","scores":{"qualidade":0.9,"prazo":0.7}},
        {"id":"B","scores":{"qualidade":0.7,"prazo":0.95}},
        {"id":"C","scores":{"qualidade":1.0,"prazo":1.0},
         "constraints":[{"id":"habilitacao","satisfied":false}]}
      ]
    }}}
  }]
}
```

Os valores são utilidades comparáveis: maior é melhor. Converter custo em utilidade é responsabilidade do modelo de decisão; não envie preço bruto supondo minimização automática. A soma ponderada é 0,82 para A e 0,80 para B; C é bloqueado apesar das pontuações altas. Pesos devem somar 1, com tolerância 1e-10, sem normalização silenciosa. O procedimento avalia, sem criar decisão persistente. Fonte: deliberation::Evaluate/execute, kernels.

Procedimento	Contrato
`phronesis.evaluate`	`evaluation`; MCDA, restrições, Pareto e sensibilidade
`phronesis.grounded`	`arguments,attacks,context?`; IDs de argumentos e referências disponíveis
`phronesis.create`	`situation,domain,evaluation,evidence_ids,policy_version,model_version,reviewers`; readers/idempotência opcionais
`phronesis.transition`	`node_id,expected_revision,to,reason`; campos específicos da etapa

Procedimento	Contrato
`phronesis.explain`, `phronesis.replay`	`node_id,context?`; explicação e replay determinístico
`phronesis.cases`	Entrada Search; restringe kind=case

Create e transition escrevem. Evaluate, grounded, explain, replay e cases são leitura. Create deve citar memórias reais disponíveis, por exemplo a origem politica-compras/v1 registrada anteriormente. Não substitua evidence_ids por IDs inventados. A decisão fixa versões de política/modelo e revisões/hashes de evidência. Mudanças dessas evidências bloqueiam transições posteriores até nova deliberação.

O ciclo é DRAFT → EVIDENCE_GATHERED → ALTERNATIVES_EVALUATED → HUMAN_REVIEW → APPROVED/REJECTED/NEEDS_MORE_EVIDENCE. O ramo aprovado continua EXECUTED → OUTCOME_RECORDED → REVIEWED. Aprovar exige revisor designado com ADMIN, review_attestation explícito e selected_alternative elegível; a API não comprova que o chamador é humano. EXECUTED registra execution_receipt, sem contratar fornecedor nem executar ação externa. O desfecho exige objeto com source_id. Cases exige memória kind=case ligada a decisão e eventos OUTCOME_RECORDED existentes e autorizados.

Limites: 256 alternativas, 64 critérios, 64 conjuntos de sensibilidade e 64 restrições por alternativa. Grounded implementa argumentação abstrata pelo menor ponto fixo; não equivale a ASPIC+ completo. Similaridade entre casos não demonstra causalidade, aplicabilidade clínica ou conformidade jurídica.

10 2 Regras Datalog JSON

`datalog.evaluate` exige feature datalog e GRAFO_READ. Sua entrada é JSON tipado, sem parser textual Datalog. A relação base edge/2 deriva das arestas do snapshot autorizado. Exemplo mínimo, sem gravar fatos derivados:

```
{"statements":[{
  "query":"CALL datalog.evaluate($request)",
  "parameters":{"request":{
    "rules":[{
      "head":{"predicate":"reachable","terms":[{"kind":"variable","name":"x"},
{"kind":"variable","name":"y"}]},
      "body":[{"predicate":"edge","terms":[{"kind":"variable","name":"x"},
{"kind":"variable","name":"y"}]}]
    }],
    "predicate":"reachable"
  }}
}]}
```

Essa regra copia arestas diretas; para fecho transitivo, acrescente reachable(x,y) ← edge(x,z), reachable(z,y). Implementa joins positivos, recursão semi-naive e negação estratificada. Variáveis da cabeça e de literais negados precisam estar vinculadas positivamente. Limites: cem regras, dezesseis literais por corpo, oito termos por átomo e mil resultados. Constantes não aceitam null, arrays ou objetos. Erros incluem `DATALOG_UNSAFE_RULE`, `DATALOG_ARITY_MISMATCH`, `DATALOG_INVALID_REQUEST` e `RESOURCE_DATALOG_RULE_LIMIT`. Fonte: datalog::Request/execute.

11 Rotinas NPL

NPL executa um subconjunto procedural próprio via `POST /db/npl`. Autenticação usa os mesmos cabeçalhos nativos; database no corpo, se presente, deve coincidir com X-Database. A identidade não pode ser fornecida pelo JSON. Fontes: npl_http::handle, NplDispatcher::dispatch.

Registre uma função e depois chame-a com o mesmo usuário:

```
{"operation":"register","name":"dobro","source":"CREATE FUNCTION dobro(x FLOAT) RETURNS FLOAT BEGIN
RETURN x + x; END;"}
```

```
{"operation":"call","query":"CALL dobro(2);"}
```

O retorno contém status, database, routine, version e resultados[0].value. Registrar exige GRAFO_WRITE; chamar exige GRAFO_READ e proprietário exato da rotina. ADMIN não ignora essa última verificação. Não há concessão EXECUTE implementada neste caminho. Fonte: NplService::register/call.

operation	Campos/efeito
register	name/source; fonte até 128 KiB; registro versionado
call	query com CALL de rotina
execute_cypher	query/parameters objeto; leitura parametrizada
query_nodes	label; property/value/limit opcionais
scan_label	label; limit opcional
memory_stats	ADMIN; estatísticas de reservas
begin, commit, rollback, write_cypher	Explicitamente recusadas antes de mutação

Funções aceitam parâmetros e retorno, DECLARE, condições IF/THEN/END IF e RETURN conforme o subconjunto implementado. `EXECUTE CYPHER ... USING ... INTO ...` usa o host nativo de leitura; não autoriza Cypher arbitrário de escrita. Exemplo direto:

```
{"operation":"execute_cypher","query":"MATCH (n:Fornecedor) WHERE n.status = $status RETURN n LIMIT
2","parameters":{"status":"habilitado"}}
```

Cada dispatch estabelece prazo de cinco segundos, mil linhas e cem mil passos, além de limites do governador. O catálogo padrão é <data_root>/_npl/catalog.json, alterável por NEXUSDB_NPL_CATALOG, com dispatcher único por processo. Memory_stats mede reservas estimadas, não todo o RSS/VRAM. HTTP retorna 401 para autenticação, 403 para autorização/proprietário, 404 para ausência, 422 para prazo/orçamento, 503 para inicialização do catálogo e 400 para outras entradas inválidas. O campo error é textual nesse endpoint.

12 Agent MCP e grafos executáveis

12 1 Ferramentas Agent

`GET /agent/tools` descobre ferramentas com X-Database; `POST /agent/tool/<nome>` executa com input e budget opcionais. Database pode vir no cabeçalho ou corpo, devendo coincidir quando ambos existem. Discovery e execução exigem READ. Fontes: agent/api, tools::builtins, executor.

Ferramenta	Entrada
database.stats, database.schema	{}
graph.bfs	source; max_depth/limit opcionais

Ferramenta	Entrada
graph.neighbors	source; limit opcional
graph.shortest_path	source/target; directed opcional
algo.pagerank	iterations/damping/limit opcionais
vector.search	index/vector; k/ef opcionais
spatial.nearby	x/y/radius; label/limit opcionais

Todas são READ_ONLY; não há escrita ou Cypher livre. Campos desconhecidos são recusados. BFS é não direcionada, inclui origem e admite profundidade até 32; neighbors exclui origem. Menor caminho não usa pesos. Busca vetorial exige índice existente e dimensão compatível. Nearby utiliza unidades das coordenadas, sem promessa de metros ou geodésia.

Exemplo para POST /agent/tool/graph.shortest_path:

```
{"input":{"source":1,"target":2,"directed":false},"budget":{"timeout_ms":5000,"max_result_rows":20}}
```

Orçamentos explícitos somente reduzem limites do servidor. Defaults Agent: dez segundos, dois jobs, cem linhas, 1 MiB de resultado, cinquenta mil nós, duzentas mil relações e 64 MiB estimados. Stats/schema dispensam a admissão por tamanho do grafo, mas mantêm autorização, concorrência, timeout e saída. Erros possuem `{status:"error",error:{code,message}}`; HTTP inclui 400/401/403/404/413/422/503/504. Timeout sinaliza cancelamento; trabalho nativo pode continuar até observar um checkpoint, mantendo sua vaga.

12 2 MCP nativo e MCP de conhecimento

O `/mcp` Rust usa ToolRegistry → ToolExecutor → funções nativas no mesmo processo; é desabilitado por padrão. Configure antes de iniciar:

```
$env:NEXUSDB_MCP_ENABLED = 'true'
$env:NEXUSDB_MCP_CONFIG = 'E:\nexusDB-tests\mcp-compras.json'
```

Conteúdo do catálogo, para database e conta previamente criados com READ:

```
{"databases":{"COMPRAS":{"users":["compras_leitor"],"tools":
["nexus_schema","nexus_graph_stats","nexus_neighbors","nexus_shortest_path"]}}}
```

Catálogo restringe publicação; não concede RBAC. Sem catálogo, usuários READ descobrem as ferramentas nativas disponíveis. Com catálogo, databases ausentes não publicam ferramentas; users=[] não permite ninguém. Mudanças exigem reinício. As oito ferramentas são nexus_schema, nexus_graph_stats, nexus_bfs, nexus_neighbors, nexus_shortest_path, nexus_pagerank, nexus_vector_search e nexus_spatial_nearby. Fonte: mcp, contrato de transporte.

Cada requisição usa X-User/X-Pass/X-Database. Initialize devolve Mcp-Session-Id; chamadas seguintes enviam sessão e MCP-Protocol-Version negociado. Depois de notifications/initialized, use tools/list, tools/call ou ping; DELETE /mcp encerra. POST exige Content-Type application/json e Accept application/json, text/event-stream. Sessão não é credencial; permissões são revalidadas e revogação READ bloqueia chamadas posteriores. Há cancelamento cooperativo por notifications/cancelled. Não há OAuth, GET SSE ou JSON-RPC em lote. Origin é recusado por padrão, salvo allowlist exata. Auditoria omite argumentos e segredos.

O adaptador Python knowledge_mcp.py é separado: stdio, sete ferramentas fixas de leitura knowledge, configuradas por NEXUSDB_MCP_URL/USER/PASSWORD/DATABASE. Não confundir seu transporte com `/mcp` nativo nem anunciar escrita por essa ponte.

12 3 GraphRAG e orquestração

O GraphRAG nativo exporta `graphRAG.retrieve(index,vector,k?,ef?,depth?,limit?)`, `graphRAG.context(start,depth?)` e `graphRAG.prompt(start,depth?)`. Retrieve combina seeds vetoriais e expansão outgoing na mesma revisão, com depth≤8 e limit 1..1000, sinalizando truncamento. Context reúne dados; prompt monta texto, sem chamar LLM. Similaridade não prova relação ou causalidade. Fontes: graph_rag, vector_routes::handle, graph_routes.

O núcleo execution oferece nós Splitter/Worker/Code/Gate/Human, arestas dependency/correction/learning/approval e restrições aprendidas. ExecutionGraph::ready_nodes calcula ondas; run executa nós elegíveis em threads. A fachada NexusDB oferece put_execution_graph, get_execution_graph, run_execution_graph e approve_execution_node. ExecutorRegistry recebe adaptadores NodeExecutor explícitos; não há executor padrão de shell ou LLM. Gates usam subconjunto determinístico de JSON Schema. Fonte: execution, EXECUTION_GRAPHS.

Para validar localmente, em dados isolados:

```
cargo test --test npl_runtime
cargo test --test agent_runtime --test mcp_runtime
cargo test --features phronesis,datalog,embedded,spatial-geoparquet --test knowledge_runtime --test
discovery_runtime
py -m unittest discover -s tests -p test_knowledge_mcp.py -v
py -m unittest discover -s tests -p test_discovery_http.py -v
```

Esses comandos correspondem a testes existentes; não foram executados nesta revisão documental. Permanecem fora do contrato: RDF/SPARQL/OWL/SHACL, ontologia/ABAC completos, ANN/FTS incremental scoped, transações NPL genéricas, execução externa automática de decisões e homologação universal de clientes MCP.

13 Automações e auditoria

Automações registram tarefas e confirmações na caixa de saída persistente do banco. Use `POST /db/admin/automation`, credenciais de administrador global e `X-Database`. O recurso exige servidor único e banco compatível com gravação; HA retorna HTTP 409 `AUTOMATION_REQUIRES_SINGLE_SERVER` e `disk_only` retorna `DISK_ONLY_UNSUPPORTED`.

```
{
  "operation":"create",
  "name":"revisar-medicao",
  "trigger":{"type":"node_condition_entered","label":"Medicao"},
  "condition":{"property":"valor","operator":"gt","value":100},
  "action":{"task":"revisar_medicao","message":"Revisar medição"},
  "notification":{"recipient":"equipe-revisao","message":"Tarefa registrada"}
}
```

`node_written` avalia cada gravação. `node_condition_entered` compara o estado anterior à transação com o estado final: uma passagem de 98 para 120 dispara; de 120 para 125 não cria novo disparo por entrada. Condições aceitam propriedades de primeiro nível e operadores `eq`, `gt`, `gte`, `lt`, `lte`; propriedade ausente não satisfaz. A criação da primeira regra ativa persistência SQLite autoritativa e diário de recuperação naquele banco.

13 1 Horários e recorrência

Para um horário único, use data legível com fuso explícito ou timezone regional:

```
{"type":"at","at":"2026-11-20T08:30:00","timezone":"America/Sao_Paulo","max_delay_ms":900000}
```

Também aceita RFC 3339 com `Z` ou offset, como `2026-11-20T08:30:00-03:00`. `unix_ms` continua disponível para integrações anteriores. Acrescente `node_id` e condição quando a tarefa depender das propriedades atuais de um nó. Sem nó e condição, a tarefa pode funcionar como lembrete independente.

```
{"type":"recurring","frequency":"weekly","start_at":"2026-11-23T08:30:00","timezone":"America/
Sao_Paulo"}
```

As frequências são `daily`, `weekly` e `monthly`. O fuso regional conserva o horário local durante mudanças de offset. Ocorrências futuras em horário inexistente são puladas; em horário duplicado vale a primeira. A agenda mensal conserva o dia original e usa o último dia em meses mais curtos. Após parada, somente a ocorrência vencida mais recente é avaliada. `max_delay_ms` pode expirar essa ocorrência, preservando a próxima.

13 2 Consulta auditoria e consumo

O agendador verifica aproximadamente a cada segundo e processa até 100 vencidos por banco/ciclo. Não oferece execução de tempo real. Consulte `next_run_at` para data legível e `due_ms` para representação numérica. Estados incluem `pending`, `executed`, `skipped` e `expired`.

```
{"operation":"list","after_id":0}
{"operation":"outbox","after_id":0}
{"operation":"audit","after_id":0}
{"operation":"delete","id":1}
```

As consultas retornam até 100 registros por página; use `next_after_id` na próxima requisição. Exclusão remove a regra e cancela o agendamento pendente, preservando registros anteriores. `property_changed` registra valores antigo e novo; campos de presença distinguem ausência de `null`. O gatilho `login` depende de login explícito bem-sucedido em `/cluster/login`; autenticação rotineira não o dispara. `database_started` passa a valer na próxima inicialização.

A caixa de saída não envia e-mail, webhook ou relatório por conta própria. Um consumidor externo deve ler registros e deduplicar entregas por ID. Exportação lógica do grafo não inclui regras, outbox ou auditoria. Fontes: contrato de automações, implementação, validação da rota administrativa.

14 Persistência backup e alta disponibilidade

A ferramenta `nexus-backup.cmd` executa `backup_app/nexus_backup.py` e requer Python 3.10+. Para incrementais e PITR, habilite `NEXUSDB_RECOVERY_JOURNAL=1`. Antes da primeira ativação em banco legado, encerre o servidor graciosamente para consolidar buffers. O modo usa SQLite autoritativo e `synchronous=FULL`, com custo adicional de I/O. Uma vez instalado no banco, o diário continua ativo mesmo removendo a variável. Não use binário antigo para gravar nesse banco.

```
nexus-backup.cmd backup --type full --data-root D:\nexusdb\data --database vendas --repo E:
\backups\vendas
nexus-backup.cmd backup --type incremental --data-root D:\nexusdb\data --database vendas --repo E:
\backups\vendas --parent ID_ANTERIOR
nexus-backup.cmd verify --repo E:\backups\vendas --backup ID_FINAL
nexus-backup.cmd list --repo E:\backups\vendas
```

O full usa SQLite Online Backup e SHA-256; copiar apenas o arquivo SQLite com WAL ativo não equivale ao procedimento. Para banco legado sem diário, somente full é permitido: use `--offline` com origem parada ou `--flush-url` para o mesmo nó local de `--data-root`. Escritas concorrentes posteriores ao flush podem ficar fora da captura.

14 1 Incrementais e PITR

Incrementais exigem todos os ancestrais, desde o full. Mudança de encarnação, DROP/recriação ou instalação de snapshot exige novo full. Não combine cadeias de nós diferentes após failover.

```
nexus-backup.cmd archive --data-root D:\nexusdb\data --database vendas --repo E:\backups\vendas --
parent ID_FINAL --interval 30
nexus-backup.cmd restore --repo E:\backups\vendas --backup ID_FINAL --at "2026-11-10T15:30:00-03:00"
--destination D:\recuperado\data\vendas
```

`archive` permanece ativo; `--once` captura um ciclo. Nenhum agendamento é instalado automaticamente. Intervalo de 30 segundos não garante RPO de 30 segundos. O diário não é podado automaticamente: monitore espaço e preserve os ancestrais das cadeias retidas.

O destino do restore não pode existir. A ferramenta reconstrói e valida uma cópia temporária antes de publicar a pasta. Inicie uma instância SINGLE isolada com o novo diretório raiz, configure o administrador separadamente e valide consultas antes de apontar clientes. O banco restaurado inicia nova identidade de diário e precisa de novo full.

PITR exige ISO 8601 com fuso e aplica transações inteiras até o timestamp lógico indicado. Falha fora da cobertura; não recupera períodos anteriores à ativação nem alterações não arquivadas. O timestamp não é o instante exato da confirmação HTTP ou de consenso HA.

O backup é por banco. Usuários, senhas, grants, catálogo `system/nexusdb_config.sqlite`, configuração de serviços e estado de cluster ficam fora. Reaplique definições de índices de propriedades mantidas apenas em memória. Incrementais cobrem nós e relacionamentos; metadados administrativos não possuem replay incremental completo. O restore não instala automaticamente dados nos demais membros HA. Os pacotes são comprimidos, sem criptografia ou assinatura; SHA-256 detecta corrupção, mas não autentica origem hostil.

14 2 Replicação e recuperação de nós

HA é experimental: heartbeat, eleição simples, log JSONL, catch-up e snapshots não equivalem a Raft completo. `NEXUSDB_MODE=HA` ou `CLUSTER` habilita o modo; configure IDs distintos, `NEXUSDB_HA_PEERS` separado por vírgulas e segredo de pelo menos 32 caracteres. Heartbeat padrão: 1000 ms; eleição: `3500 + server_id × 250` ms. Quorum estrito, leitura de réplicas e autenticação de cluster são habilitados por padrão. Mantenha tráfego em rede privada ou túnel TLS e configure catálogo de segurança em cada nó.

Fontes: backup e restore, ferramenta, diário, HA.

15 Gateway e observabilidade

O gateway publica operações aprovadas com chaves próprias. Compile-o separadamente e configure conta upstream com READ no banco. Ele não recebe automaticamente as credenciais do instalador.

```
cargo build --manifest-path crates/nexusdb-rest-gateway/Cargo.toml
```

Variável	Padrão ou exigência
NEXUSDB_GATEWAY_CONFIG	endpoints.json
NEXUSDB_GATEWAY_STATE	gateway-control.sqlite
NEXUSDB_GATEWAY_BIND	127.0.0.1:8787
NEXUSDB_GATEWAY_ADMIN_KEY	Obrigatória e privada, com pelo menos 32 bytes
NEXUSDB_BASE_URL	Origem upstream configurada pelo operador

15 1 Publicação e chaves

NEXUSDB_BASE_URL deve ser uma origem HTTP(S) sem usuário, senha, query, fragmento ou caminho além da raiz. Upstreams remotos exigem HTTPS; HTTP é aceito somente nos hosts de loopback reconhecidos. Configuração inválida impede a inicialização com INVALID_UPSTREAM_ORIGIN ou REMOTE_UPSTREAM_REQUIRES_HTTPS. Chave administrativa curta retorna ADMIN_KEY_TOO_SHORT.

As credenciais upstream são obtidas das variáveis referidas pelos perfis. Alteração de perfil, conta ou configuração requer reinício. Use diretório privado gravável para o estado SQLite. Os perfis `nodes`, `relationships`, `query` e `call` fixam banco, operação, campos e parâmetros; o consumidor não envia consulta arbitrária nem substitui credenciais.

```
POST /admin/endpoints/leitos/keys
X-Gateway-Admin-Key: CHAVE_ADMIN
{"ttl_seconds":3600}

POST /v1/endpoints/leitos
X-API-Key: CHAVE_EMITIDA
{"params":{"sector":"A"}}
```

Bearer é alternativa de autenticação do gateway, não da porta nativa 7474. Uma chave autoriza um endpoint e dura de um segundo a 365 dias. Alterar o perfil invalida chaves anteriores com `API_KEY_PROFILE_CHANGED`; somente trocar a senha da mesma conta upstream não modifica o vínculo do perfil.

`POST /admin/access` controla scopes `key`, `endpoint`, `database` ou `global`. `/admin/state`, `/admin/audit` e `/openapi.json` exigem chave administrativa. O OpenAPI de um endpoint exige sua chave válida. `/health` é público e indica apenas o processo gateway, sem comprovar conectividade do banco.

15 2 Limites e observabilidade

Os limites incluem 64 KiB por request, 4 MiB de resposta upstream e 16 chamadas simultâneas. Há taxa por chave e limites por banco. Excesso pode retornar `RATE_LIMIT`, `GATEWAY_BUSY` ou `DATABASE_BUSY`. Outros erros úteis: `API_KEY_INVALID`, `API_KEY_EXPIRED`, `API_KEY_REVOKED`, `PARAM_REQUIRED`, `PARAM_TYPE_OR_RANGE`, `UPSTREAM_PERMISSION_DENIED` e `UPSTREAM_TIMEOUT`.

Cancelamento é cooperativo: não preempta locks, chamadas bloqueantes ou kernel CUDA em andamento. O gateway revalida autorização após resposta upstream e descarta dados quando acesso foi revogado durante a execução. Dados já enviados não podem ser recolhidos. Bloquear o gateway não bloqueia a porta nativa.

`/cluster/status`, `/cluster/metrics` e `/cluster/health` fornecem estado HA. As métricas incluem papel, termo, índices, tamanho do WAL e contadores de peers, não monitoramento completo de CPU/RAM. Reservas gerenciadas não medem todo RSS ou VRAM. Não confunda health check com garantia de consistência. Não há confirmação de rotas `/health/live` e `/health/ready` apenas por aparecerem como preferência nas instruções do repositório.

Comandos de validação disponíveis, a executar em ambiente descartável:

```
cargo test --test password_security_runtime
cargo test --manifest-path crates/nexusdb-rest-gateway/Cargo.toml
python -m unittest discover -s backup_app -v
python backup_app/integration_test.py --server target/release/nexusdb.exe
py examples/rest_gateway_validation.py --output E:\nexusDB-tests\gateway-validation
```

Estes comandos são referências operacionais; sua inclusão não afirma execução nesta revisão. Fontes: gateway, contratos v0.3, rotas e erros, métricas HA.

16 Laboratórios e integração de dados

16 1 Grafo comercial mínimo

Crie EXAMPLE_MANUAL com uma conta administrativa e use o cliente Python do capítulo 3. Este laboratório estabelece a sequência de criação, correlação e consulta sem pressupor IDs fixos.

```
post("/db/admin/database/create",
     {"database": "EXAMPLE_MANUAL"})

created = post("/db/data/node/bulk", {"rows": [
    {"key": "C001", "label": "Cliente",
     "properties": {"codigo": "C001", "nome": "Ana"}},
    {"key": "P01", "label": "Produto",
     "properties": {"sku": "P01", "nome": "Café"}},
]})
ids = {row["key"]: row["id"]
       for row in created["resultados"]}

post("/db/data/relationship/bulk", {"rows": [{
    "key": "compra-1",
    "from": ids["C001"],
    "to": ids["P01"],
    "type": "COMPRA",
    "properties": {"quantidade": 2},
}]})
```

```
result = query(
    "MATCH (c:Cliente)-[r:COMPRA]->(p:Produto) "
    "RETURN c.codigo, p.sku, r.quantidade"
)
print(result["resultados"])
```

O resultado esperado contém C001, P01 e a quantidade 2. O campo key correlaciona resposta e entrada. Uma segunda execução completa cria novos dados; use um banco novo para repetir a carga integral.

Para modelar vendas com vários itens, crie Pedido como nó intermediário. Guarde a quantidade e o subtotal na relação CONTEM. Acrescente NO_DIA quando a aplicação precisar percorrer uma dimensão de calendário.

16 2 Análises temporais e machine learning

Defina a unidade de análise antes de exportar dados. Um registro de venda, uma janela de sete dias e uma série mensal de produto são unidades diferentes. Uma ordenação cronológica não torna qualquer algoritmo sensível à sequência.

A aplicação pode extrair dados autorizados, agregar em Python e registrar resultados como nós Janela e ClusterTemporal. Relacione a janela ao produto, ao dia inicial e ao cluster da execução. Armazene a versão do método, os parâmetros e a origem do dataset.

Mantenha atributos de previsão anteriores ao alvo. Separe treino e teste cronologicamente quando houver dependência temporal. Um resultado de clustering não comprova estabilidade, causalidade ou desempenho de previsão.

MATCH não oferece agregação geral por mês com SUM e múltiplos saltos. Use consultas compatíveis, extração em lote ou Arrow e calcule a agregação na aplicação. Esse limite deve aparecer na metodologia de uma comparação.

16 3 Rede social

Em EXAMPLE_SOCIAL, represente Pessoa, Grupo, SEGUE e PARTICIPA. Use as rotas de lote para obter IDs e criar arestas. Compare grau, componentes conectados e PageRank.

```
CALL algo.degree() LIMIT 20
CALL algo.connectedComponents()
CALL algo.pageRank(20) LIMIT 20
```

A centralidade mede a posição no grafo sob as regras do algoritmo. Não é uma classificação de qualidade de uma pessoa. Anote se o algoritmo percorre relações dirigidas e quais projeções compõem o grafo analisado.

16 4 Proveniência científica e revisão

Use bancos separados para dados brutos e memórias Knowledge quando os perfis de acesso forem distintos. No grafo, represente Amostra, Medicao e Pesquisador. Em Knowledge, registre conteúdo, origem e evidências autorizadas. Na camada execution, registre adaptadores de análise e gates para validar saídas.

O grafo executável registra aprovação e evidência de execução. Phronesis registra alternativas, restrições e revisão. A aplicação executa ações externas e fornece recibos. Confira os capítulos 10 e 12 antes de conectar uma decisão a um processo real.

16 5 RAG modular

Uma integração RAG pode ter recuperadores independentes para documentos, grafo e histórico. O orquestrador seleciona o recuperador, monta um contexto limitado e envia evidências citáveis ao gerador.

knowledge.contextPack aplica o recorte das memórias; graphRAG.context expande uma vizinhança; graphRAG.temporalRetrieve usa projeção e estado histórico quando solicitado. Registre quais evidências vieram de um snapshot local e quais foram consultadas durante a pergunta.

O gerador de linguagem pertence à aplicação. Envie identificadores de evidência e exija citações e indicação de lacunas. Texto recuperado deve ser tratado como dado, inclusive quando contiver instruções.

Referências — exemplos, GraphRAG, Knowledge e exportação Arrow.

17 Diagnóstico configuração e manutenção

17 1 Sequência de diagnóstico

Comece por endereço, porta, executável e banco. Confira o processo que está ouvindo, o data root e as features compiladas. HTTP 401 antes da criação do banco indica autenticação. Database not found exige corrigir o nome ou criar explicitamente o banco.

Para Phronesis, a sequência é compilar com a feature, iniciar esse executável, criar o banco de demonstração e executar knowledge.initialize com uma conta administrativa. São condições independentes.

Código ou mensagem	Causa a verificar	Ação
KNOWLEDGE_FEATURE_DISABLED	Executável sem a feature	Compile e inicie o binário apropriado
KNOWLEDGE_POLICY_REQUIRED	Política ausente	Inicialize por ADMIN no banco escolhido
Database not found	Nome inexistente na instância	Confira X-Database e crie o banco
KNOWLEDGE_SCOPED_DATABASE	Acesso bruto fora do contrato	Use as procedures scoped autorizadas
TEMPORAL_HISTORY_DISABLED	Histórico não ativado	Ative por banco antes de consultar versões
TEMPORAL_HISTORY_ALREADY_ENABLED	Tentativa de desligar histórico	Preserve a continuidade do histórico
DISK_ONLY_UNSUPPORTED	Operação exige memória	Use a API compatível com o modo em disco
COMPUTE_BUSY	Admissão saturada	Reduza concorrência ou ajuste limites
COMPUTE_MEMORY_LIMIT	Orçamento insuficiente	Reduza carga ou dimensione memória
COMPUTE_SNAPSHOT_CHANGED	Revisão mudou na materialização	Reavalie consulta e concorrência
AUTH_BUSY	Limite de hashing/admissão	Aguarde e confira configuração
AUTH_RATE_LIMIT	Excesso de tentativas	Aguarde a janela de taxa
CYPHER_UNSUPPORTED_CLAUSE	Cláusula fora do subconjunto	Use sintaxe comprovada
AUTOMATION_REQUIRES_SINGLE_SERVER	Automação em HA	Use servidor único para esse recurso
API_KEY_PROFILE_CHANGED	Perfil publicado mudou	Emita uma chave para o perfil atual

A grafia e o envelope podem variar entre famílias. Registre código, endpoint, modo e versão, sem incluir senhas ou tokens. As mensagens acima não substituem o diagnóstico devolvido pela operação.

17 2 Configuração por responsabilidade

Família	Variáveis de referência
Instância	NEXUSDB_HTTP_ADDR, NEXUSDB_MODE, NEXUSDB_DATA_ROOT
Primeiro administrador	NEXUSDB_ADMIN_PASSWORD
Recuperação	NEXUSDB_RECOVERY_JOURNAL
Compute	NEXUSDB_COMPUTE_BACKEND, NEXUSDB_COMPUTE_THREADS, NEXUSDB_COMPUTE_MEMORY_MB
CUDA	NEXUSDB_CUDA_ENABLED e biblioteca configurada
Agentes	Seção agent e limites publicados no discovery
MCP	NEXUSDB_MCP_ENABLED, NEXUSDB_MCP_CONFIG
HA	NEXUSDB_HA_SERVER_ID, NEXUSDB_HA_PEERS, NEXUSDB_HA_SECRET
Segredos	NEXUSDB_SECRETS_DPAPI_FILE, NEXUSDB_PASSWORD_PEPPER_FILE
Gateway	NEXUSDB_GATEWAY_CONFIG, NEXUSDB_GATEWAY_STATE, NEXUSDB_GATEWAY_ADMIN_KEY
NPL	NEXUSDB_NPL_CATALOG
Recursos compartilhados	Seção resources e suas subclasses

Os valores são lidos por componentes distintos. Confira o arquivo de exemplo e o código responsável antes de aumentar limites. Memória gerenciada estima estruturas registradas, e não o RSS inteiro.

17 3 Verificações disponíveis

Os alvos abaixo existem no repositório. Foram executados nove exemplos representativos em uma instância isolada, cobrindo lotes, relação, MATCH, histórico, NPL, ativação Knowledge, memória e avaliação Phronesis. Essa checagem não representa execução de toda a suíte. Use as features necessárias para cada alvo.

```
cargo test --lib
cargo test --test query_runtime --test grasp_runtime
cargo test --test graph_runtime --test vector_runtime
cargo test --test temporal_history_runtime
cargo test --test temporal_projection_runtime
cargo test --test agent_runtime --test mcp_runtime
cargo test --test npl_runtime
cargo test --features phronesis,datalog,embedded,spatial-geoparquet --test knowledge_runtime --test
discovery_runtime
cargo test --manifest-path crates/nexusdb-rest-gateway/Cargo.toml
```

O teste csr_runtime_integration pertence ao alvo graph_runtime. Não há um arquivo tests/csr_runtime_integration.rs no inventário inspecionado.

17 4 Glossário

Termo	Significado
Baseline	Estado criado ao ativar histórico
CSR	Representação compacta de adjacências
Snapshot	Visão capturada de dados ou projeção
Watermark	Limite confirmado da cobertura histórica
Epoch	Identificador de uma encarnação de histórico ou feed
Projeção	Subgrafo selecionado por camadas, validade e filtros
Tempo de registro	Instante lógico da confirmação conhecido pelo banco
Validade de negócio	Intervalo declarado para a entidade ou relação
Outbox	Caixa persistente de tarefas e confirmações
Scoped	Acesso recortado por contexto e visibilidade
Pareto	Alternativas não dominadas nos critérios considerados
Context pack	Evidências recuperadas para consumo de uma aplicação
Grant	Permissão explícita no catálogo

17 5 Fontes e rastreabilidade

A revisão foi organizada com EGM Bootstrap, usando inspeção de código e revisões por área com contexto separado. Os capítulos citam arquivos, funções, rotas e flags encontrados. Os testes citados foram conferidos no inventário. Verificação estática e execução de testes têm alcances diferentes.

As fontes primárias incluem Cargo.toml, src/main.rs, src/data.rs, src/model, src/graph, src/knowledge, src/agent, src/npl_dispatch.rs, src/storage.rs, src/automation.rs e crates/nexusdb-rest-gateway. Os guias e exemplos apoiaram a leitura; divergências foram resolvidas pela implementação.

O manual anterior tinha 18 páginas e referência de setembro de 2026. Esta edição reorganiza os temas e inclui recursos posteriores. O gerador tools/create_nexusdb_manual.py corresponde à edição anterior. Reexecutá-lo pode sobrescrever o DOCX com conteúdo antigo. A fonte Markdown desta edição é a referência editorial atual.

Referências — features, servidor, configuração, segurança, backup, MCP e fonte editorial.